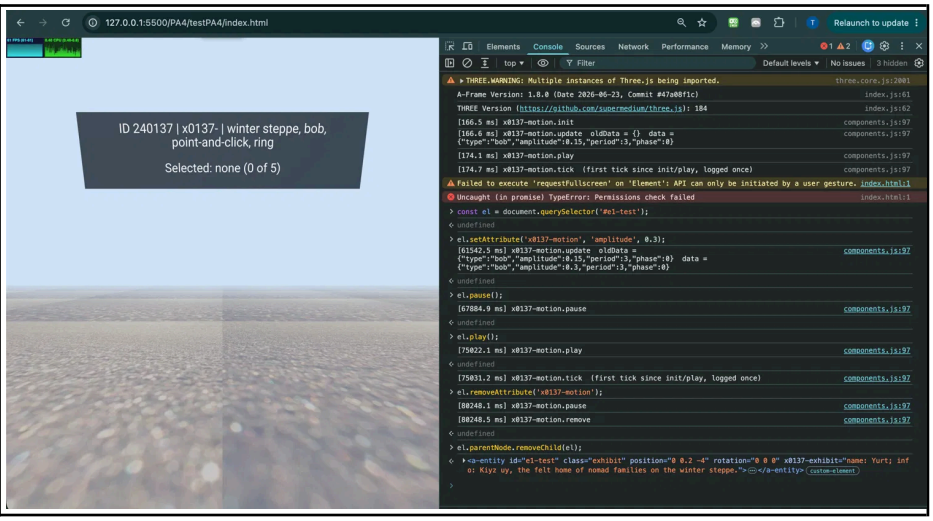
Write-up

Student: Margarita Serebrennikova
Student ID: 240137
VARIANT: winter steppe theme (last digit 7)
bob motion (3 mod 3 =0)
point-and-click interaction (1 mod 2 =1)
ring layout, 6 exhibits at radius 4m (0 mod 2 =0)
A-Frame 1.8.0 (Three.js r184)

E1 — The component life cycle

Prediction: On load: init, then update, then play and tick. setAttribute calls only update. el.pause() and el.play() call pause and play. removeAttribute calls remove. Removing the entity afterwards logs nothing, because the component is already gone.

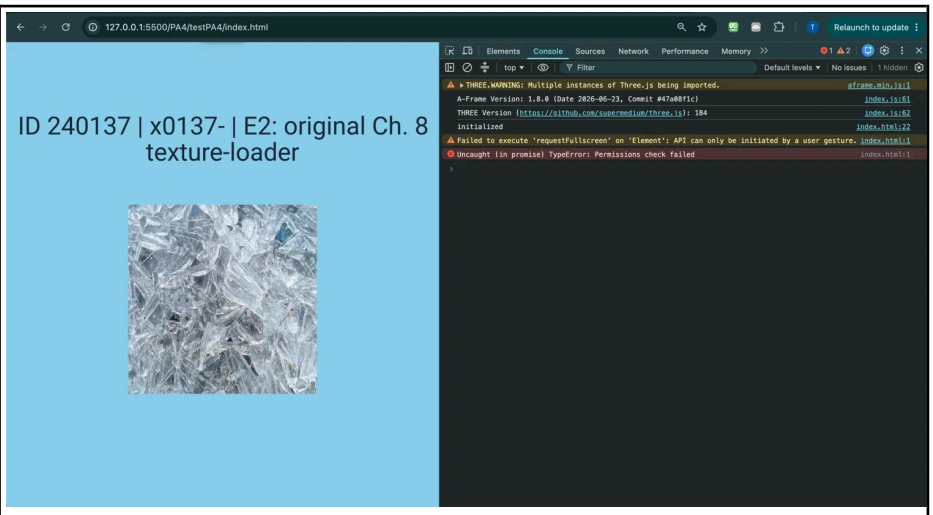
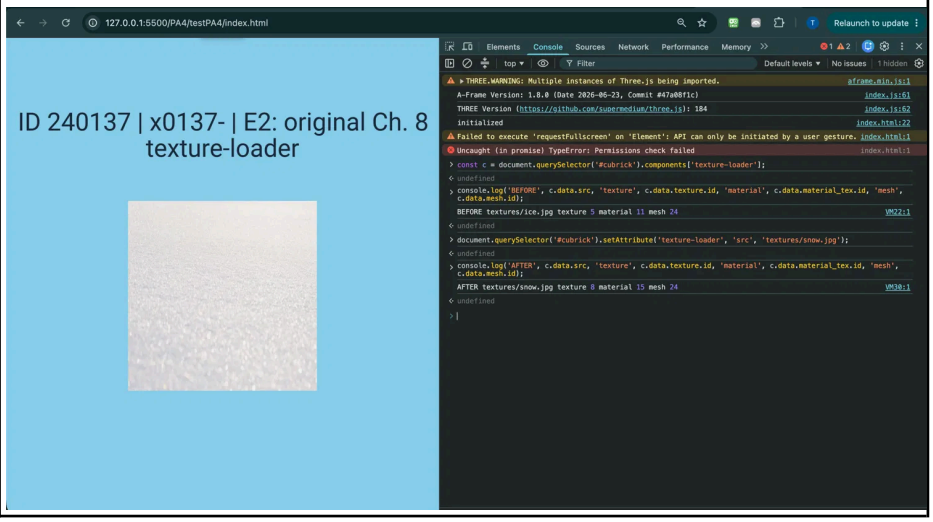
Result: ach life-cycle method of x0137-motion logs a timestamp (yurt only, id e1-test).	

Step	***Logged methods(ms)***	***oldData in Update***
Page load	init 166.5 -> update 166.6 -> play 174.1 -> tick 174.7	{}
setAttribute amplitude 0.3	update 61542.5	{type: bob, amplitude: 0.15, period: 3, phase: 0}
el.pause()	pause 67884.9	-
el.play()	play 75022.1 -> tick 75031.2	-
removeAttribute(‘x0137-motion’)	pause 80248.1 -> remove 80248.5	-
Remove the entity	Nothing (panel: 0 of 5)	-

Explanation: update fires once straight after init (0.1 ms later), with oldData = {} because there is no previous data. After that it fires only when the component’s data changes: oldData then holds previous values (amplitude 0.15) and this.data new ones (0.3), so I can compare them. tick runs only while the component is playing, pause and play switch it off and on. removeAttribute pauses the component first and then calls remove, where my code restores the base position. Removing the entity logs nothing from x0137-motion, because that component was already removed, x0137-exhibit still unregistered, so the counter dropped from 6 to 5.

E2 — Checking Ch. 8

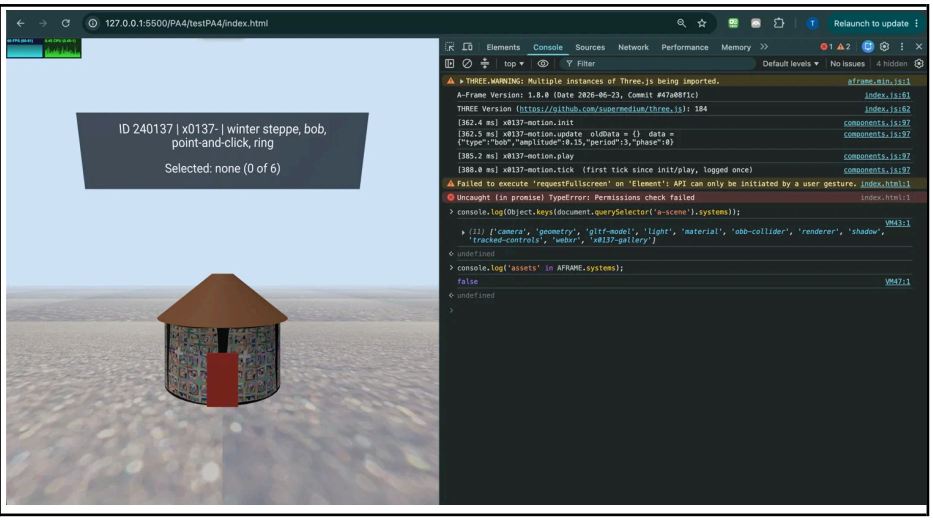
a) Prediction: setAttribute rebuilds this.data, and update creates new objects without freeing old ones

Result: *I ran the book's texture-loader unchanged on a box and changed src from ice.jpg to snow.jpg*	 

this.data field	***before***	***after setAttribute('src')***
src	textures/ice.jpg	textures/snow.jpg
texture	texture #5	texture #8 (new)
material_tex	MeshLambertMaterial #11	MeshLambertMaterial #15
mesh	Mesh #24	Mesh #24 (same)

Explanation: one setAttribute made book's update run again and build new texture and new material. The old texture #5 and material #11 were never disposed, so they stay in GPU memory, and every change leaks more. The schema describes settings that A-Frame parses from the HTML attribute; storing objects in it makes them look like settings, so getAttribute returns them and setAttribute can overwrite them. Runtime objects belong on this (this.texture, this.material, this.mesh), which A-Frame never parses, so they can be reused and disposed of in remove().

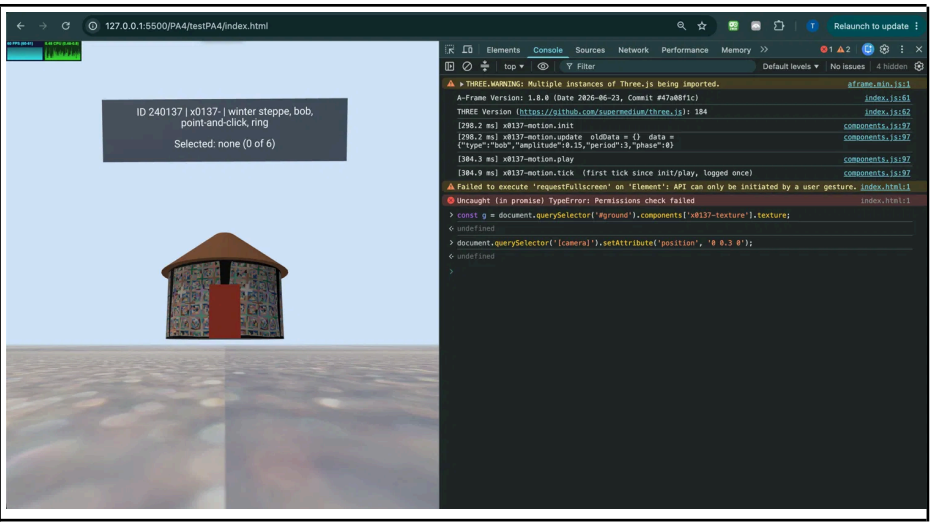
b) Prediction: <a-assets> is not a system

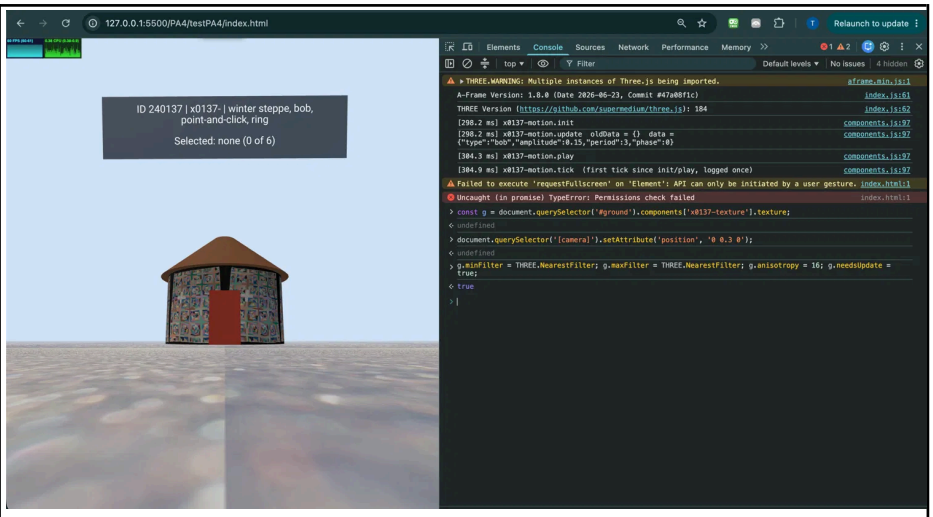
Result: Object.keys(sceneEl .systems) lists 11 systems: camera, geometry, gltf-model, light, material, obb-collider, renderer, shadow, tracked-controls, webxr and x0137-gallery, with no assets. 'assets' in AFRAME.systems returns false.	

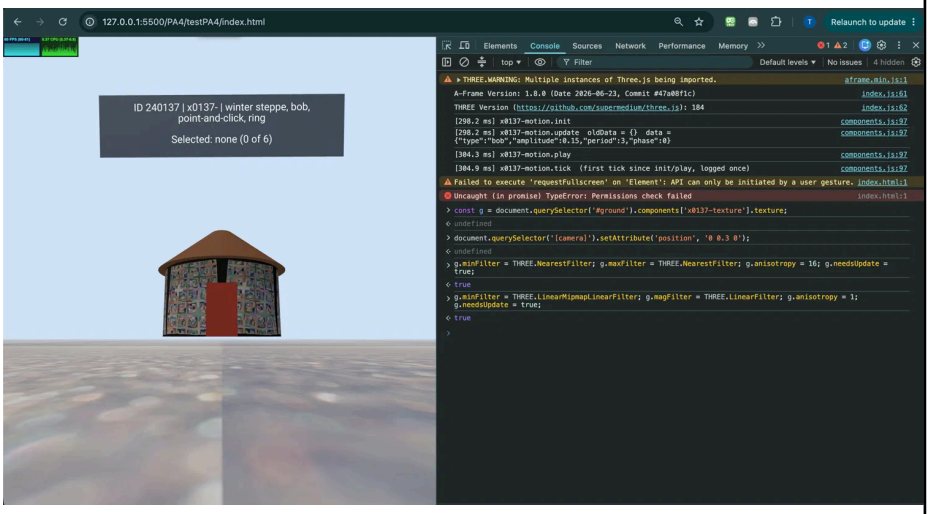
Explanation: AFRAME.registerSystem creates one object per scene, shared by every instance of a component and reachable through sceneEl.systems; my x0137-gallery appears there. <a-assets> is HTML element that preloads files and holds the scene until they are ready. It is asset management, but not an ECS system, so the book's claim does not hold.

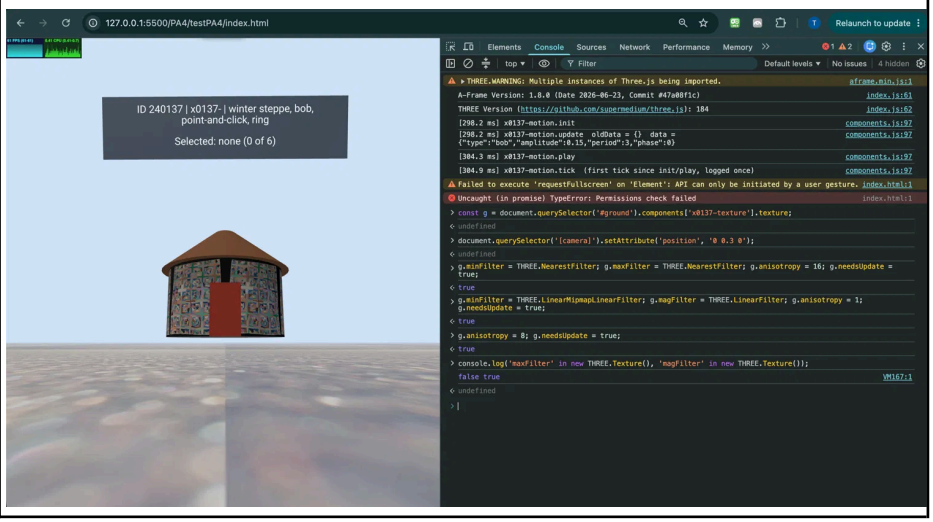
E3 — Texture filtering

Prediction: The Ch. 8 filters will shimmer most in the distance; the defaults will look smooth but blurry towards the horizon; my filters with anisotropy will stay sharpest.

Result: *Result. Same low, grazing view of the snow ground (camera at 0.3 m), three settings.*	





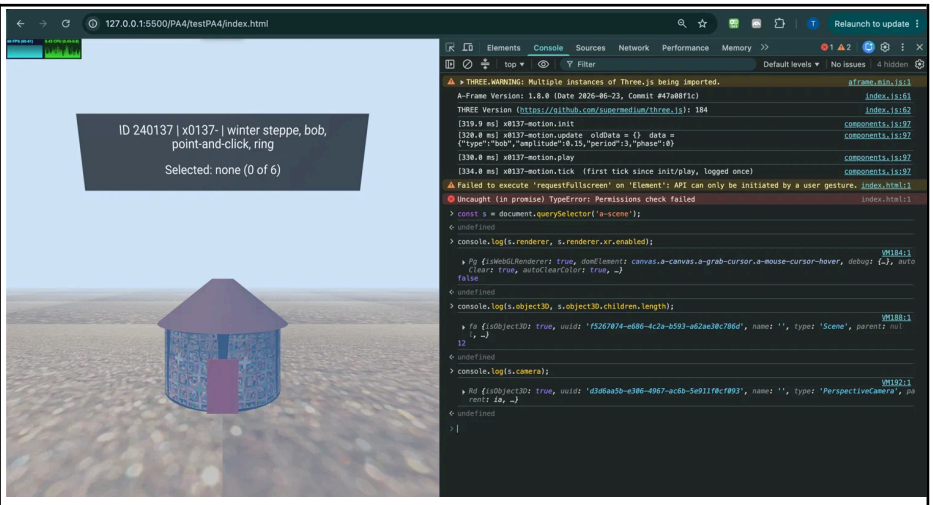


Setting	*minFilter*	*magFilter*	*anisotropy*	*Distant ground*
Ch 8	*NearestFilter*	*“maxFilter” doesn’t exist*	*16, ignored*	*grainy, noisy pixels that flicker as view moves*
three.js defaults	*LinearMipmapLinear filter*	*LinearFilter*	*1*	*smooth but the far band blurs into flat gray*
Mine	*LinearMipmapLinear filter*	*LinearFilter*	*8*	*smooth, with more detail kept just below horison*

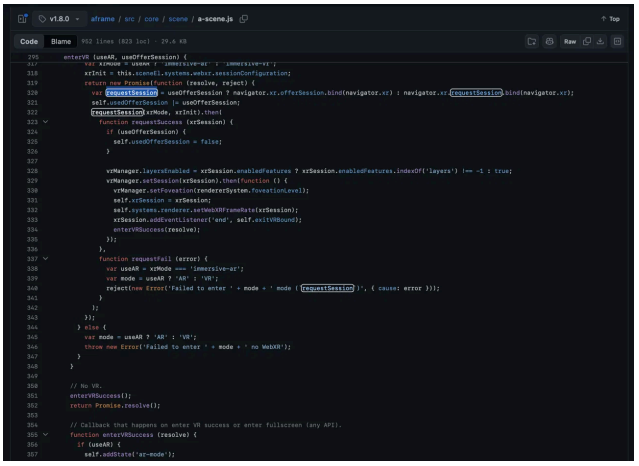
Explanation: Minification is when many texels fall into one pixel (distant ground); magnification is when one texel covers many pixels (close up). Mipmaps are pre-shrunk copies of the image at half, quarter and smaller sizes, so distant pixel samples are an averaged copy instead of one random texel. NearestFilter picks a single texel and ignores mipmaps, which causes shimmer when the camera moves. LinearMipmapLinearFilter blends between texels and between two mipmap levels. I chose 8, capped at the GPU's maximum, as balance between sharpness and cost.

E4 — What <a scene> builds

Prediction: One <a-scene> tag creates the same Three.js objects I built by hand in PA3.

Result:	
sceneEl.renderer	THREE.WebGLRenderer (isWebGLRenderer: true, draws into canvas.a-canvas); renderer.xr.enabled = false
sceneEl.object3D	THREE.Scene (type 'Scene'), 12 children
sceneEl.camera	THREE.PerspectiveCamera from my rig's camera entity

The WebXR session starts in enterVR() of <a-scene> (src/core/scene/a-scene.js, v1.8.0, line 295). Line 318 reads the session options from the webxr system. Lines 320-322 call navigator.xr.requestSession(xrMode, xrInit), where xrMode is immersive-vr or immersive-ar (line 317). Line 329 calls vrManager.setSession(xrSession), which is Three.js's renderer.xr.setSession. Line 333 adds xrSession.addEventListener('end', self.exitVRBound).

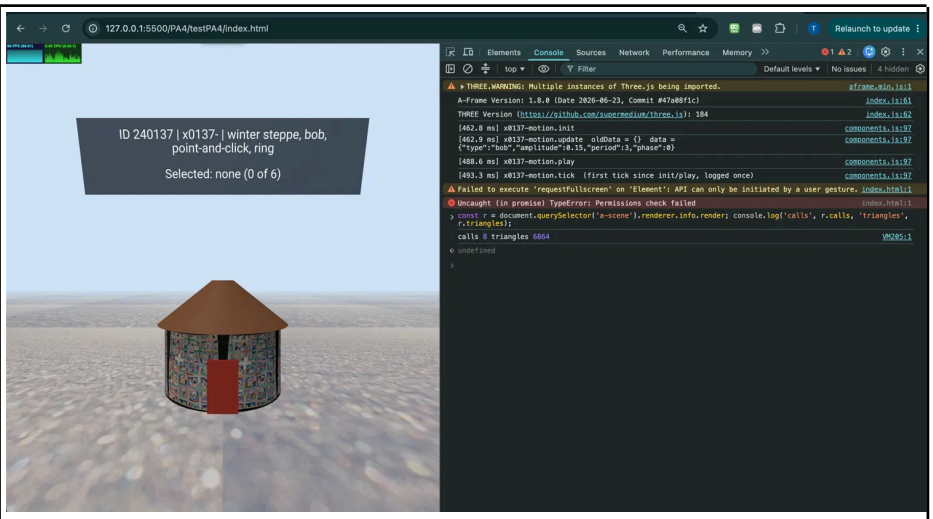
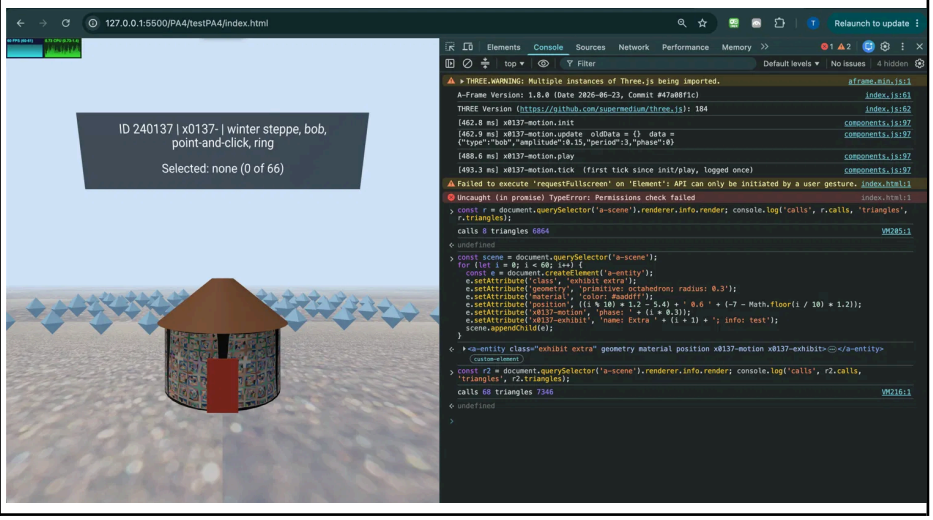


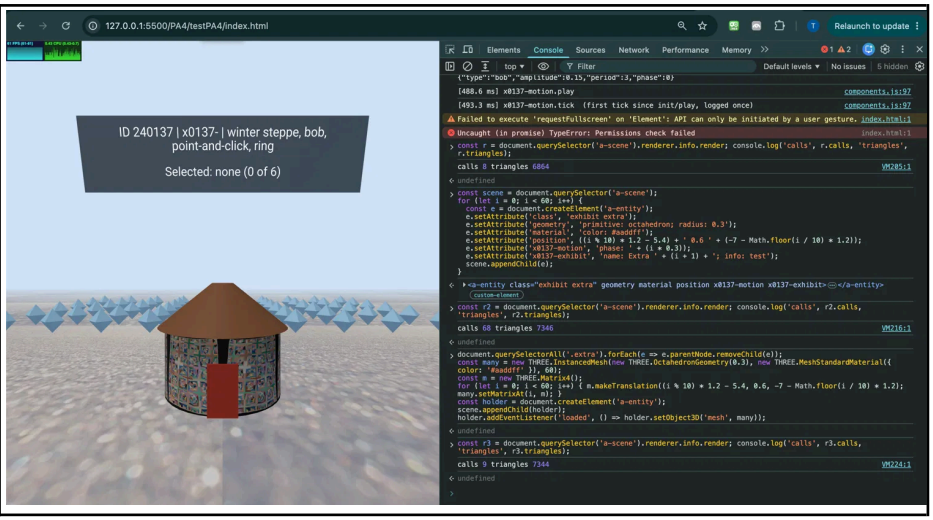
Step	PA 3 (My code)	A-frame (Enter VR)
Support check + button	*isSessionSupported + my START button*	*Enter VR button (xr-mode-ui)*
Request session	*requestSession('immersive-ar', {requiredFeatures, …})*	*I 320-322*
Connect to three.js	*renderer.xr.setSession(session)*	*I 329*
End handling	*addEventListener('end', onSessionEnd)*	*I 333*

Explanation: In PA3 I wrote this life cycle myself; A-Frame performs the same steps inside enterVR and provides the Enter VR button. Company should choose raw Three.js when it needs full control, for example custom AR shopping page with hit testing, anchors and its own session flow, or small bundle inside an existing web app. A-Frame is the better choice for a VR showroom or a teaching scene that a small team must build quickly, because HTML entities and reusable components are faster to write and change.

E5 — Performance

Prediction. Each added exhibit is one more mesh, so 60 exhibits add about 60 draw calls but few triangles, and fps may drop.

Result: fps from the stats panel; draw calls and triangles from sceneEl.renderer.info.render, because the A-Frame 1.8 stats panel shows only FPS, CPU and GPU. Same start view each time	 



Scene	*fps*	*Draw calls*	*Triangles*
My scene	*60*	*8*	*6864*
+ 60 exhibits (one entity each)	*60*	*68*	*7346*
Same 60 as one THREE.instancedMesh	*61*	*9*	*7344*

Explanation: Adding 60 exhibits added 60 draw calls (8 -> 68), one per mesh, but only 482 triangles, because each octahedron has just 8 triangles. The laptop still held 60 fps. Drawing the same 60 octahedrons as one InstancedMesh brought it back to 9 calls, with the same triangles and the same picture.

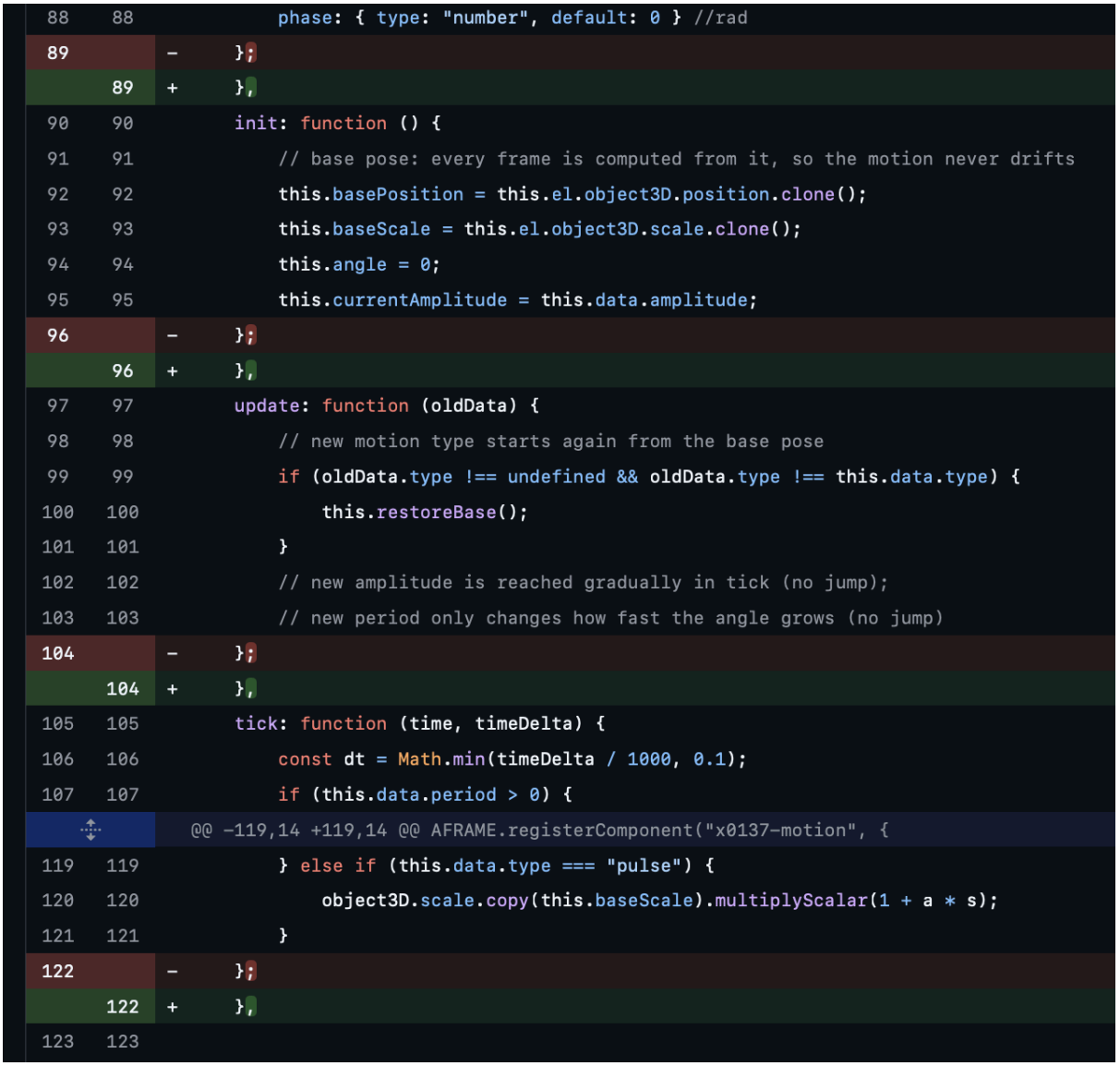
E6 — Comfort and design

Exhibits stand in a 4m ring around the start point: far enough to see a whole exhibit without tilting front view, close enough to read its panel, and none inside personal space. My text uses width 3m with 40 characters per line, so a character is about 0.075m wide. At 4m that is atan(0.075 / 4) ≈1.1°, and the ID panel at 3.5 m gives ≈1.2°, above the roughly 1° needed for comfortable reading.

In VR both controllers have a laser limited to exhibits with far: 20, so every exhibit is reachable from the centre. The main comfort risk is smooth WASD movement in VR, which causes motion sickness. I reduced it with the layout: the user can see and select all six exhibits by turning on the spot, so walking is never needed. The bob is also small and slow (0.15 m, 3 s).

Development Log

Debugging:
I mostly had Syntax errors in components.js file, writing ; instead of ,
Found through Console showing SyntaxError: Unexpected token ';' at components.js:89.
debugging syntax errors lines 89, 96, 104, 122, 129 (commit 1adae02)



```
88  88          phase: { type: "number", default: 0 } //rad
89      -   };
    89  +   },
90  90      init: function () {
91  91          // base pose: every frame is computed from it, so the motion never drifts
92  92          this.basePosition = this.el.object3D.position.clone();
93  93          this.baseScale = this.el.object3D.scale.clone();
94  94          this.angle = 0;
95  95          this.currentAmplitude = this.data.amplitude;
96      -   };
    96  +   },
97  97      update: function (oldData) {
98  98          // new motion type starts again from the base pose
99  99          if (oldData.type !== undefined && oldData.type !== this.data.type) {
100 100             this.restoreBase();
101 101         }
102 102         // new amplitude is reached gradually in tick (no jump);
103 103         // new period only changes how fast the angle grows (no jump)
104     -   };
    104 +   },
105 105   tick: function (time, timeDelta) {
106 106       const dt = Math.min(timeDelta / 1000, 0.1);
107 107       if (this.data.period > 0) {
      @@ -119,14 +119,14 @@ AFRAME.registerComponent("x0137-motion", {
119 119         } else if (this.data.type === "pulse") {
120 120             object3D.scale.copy(this.baseScale).multiplyScalar(1 + a * s);
121 121         }
122     -   };
    122 +   },
123 123
```